



.txt



.csv



.json

Archivos de Texto, CSV y JSON — Ejercicios Prácticos

Leer, escribir y gestionar archivos desde tu programa

De PSeInt a Python — Guía Práctica Paso a Paso

Docente: Jhonathan Arley Peñaloza Sierra

Skill: Introducción a Python | Campuslands · 2026

`open()` · `with` · `csv` · `json` · `read` · `write` · `append`

⚠ Antes de empezar: lee esto primero

El manejo de archivos tiene una diferencia fundamental con todo lo que has aprendido antes: el código crea, modifica y elimina archivos REALES en tu computador. Antes de empezar los ejercicios, entiende estas tres cosas:

1. ¿Dónde se crean los archivos?

Cuando escribes `open("archivo.txt", "w")`, Python crea el archivo en la misma carpeta donde está tu script `.py`. Esto se llama directorio de trabajo.

Si no ves el archivo creado, verifica en qué carpeta guardaste tu script.

```
import os
print(os.getcwd()) # muestra en qué carpeta estás trabajando
```

2. El modo "w" BORRA sin preguntar

Si abres un archivo existente con modo "w", todo su contenido se elimina de inmediato. No hay advertencia ni confirmación. Este es el error más común y más costoso en este tema.

`open("datos.txt", "w")` ← **BORRA TODO el contenido anterior**

`open("datos.txt", "a")` ← **agrega al final, conserva lo anterior**

3. Siempre usa `with open()` — sin excepción

`with` cierra el archivo automáticamente al terminar el bloque, aunque ocurra un error. Sin `with`, si tu programa falla a la mitad, el archivo puede quedar bloqueado o corrompido.

Tabla de modos de `open()` — referencia rápida

Modo	¿Qué hace?	Si el archivo YA existe	Si el archivo NO existe
"r"	Solo lectura	Abre normalmente	❌ Error (FileNotFoundError)
"w"	Solo escritura	⚠ Borra todo el contenido	✅ Lo crea
"a"	Añadir al final	✅ Agrega al final	✅ Lo crea
"r+"	Leer y escribir	Abre normalmente	❌ Error (FileNotFoundError)

Conceptos Clave: Archivos en Python

Repasa esta sección cuando tengas dudas durante los ejercicios.

Anatomía del manejo de archivos:

```
# — TEXTO (.txt) —————
with open("archivo.txt", "w") as f: # abrir en modo escritura
    f.write("Primera línea\n")      # escribir
    f.write("Segunda línea\n")

with open("archivo.txt", "r") as f: # abrir en modo lectura
    contenido = f.read()            # leer todo
    print(contenido)

with open("archivo.txt", "a") as f: # añadir al final
    f.write("Tercera línea\n")

# — CSV (.csv) —————
import csv

with open("datos.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["nombre", "edad"]) # encabezados
    writer.writerow(["Ana", 20])

with open("datos.csv", "r") as f:
    lector = csv.DictReader(f)          # lee como diccionario
    for fila in lector:
        print(fila["nombre"], fila["edad"])

# — JSON (.json) —————
import json

datos = [{"nombre": "Ana", "edad": 20}]

with open("datos.json", "w") as f:
    f.write(json.dumps(datos, indent=2)) # escribir JSON

with open("datos.json", "r") as f:
    cargado = json.load(f)              # leer JSON
    print(cargado[0]["nombre"])
```

Comparación: .txt vs .csv vs .json

	.txt	.csv	.json
¿Qué es?	Texto plano libre	Tabla con separadores	Datos estructurados
Estructura	Sin estructura fija	Filas y columnas	Objetos clave:valor
Leer con	open() + read()	csv.DictReader()	json.load()
Escribir con	open() + write()	csv.writer()	json.dumps()
Uso ideal	Notas, logs, textos	Datos tipo Excel	APIs, configuración

💡 **Regla simple:** .txt para texto libre. .csv cuando tus datos son una tabla. .json cuando tus datos tienen estructura clave-valor (como un diccionario).

PARTE 1: EJERCICIOS GUIADOS

Sigue cada paso como si estuvieras con un tutor a tu lado

Tema 1: Archivos de Texto (.txt)

Los archivos .txt son el punto de partida. Aprenderás el flujo completo: crear, escribir, leer y añadir contenido. Este flujo se repite igual para .csv y .json, así que dominarlo aquí te facilitará todo lo demás.

Ejercicio 1.1 — Flujo completo: diario personal

Enunciado

Crea un archivo llamado "mi_diario.txt". Escribe dos líneas iniciales. Luego léelo para verificar que se guardó. Después añade una línea más sin borrar lo anterior. Finalmente lee el archivo completo de nuevo. Este ejercicio te muestra el ciclo de vida completo de un archivo de texto.

Guía paso a paso:

Paso 1: Crear y escribir el archivo con modo "w".

```
with open("mi_diario.txt", "w") as f:
    f.write("Nombre: Carlos\n")
    f.write("Fecha: 2026-03-21\n")

# El archivo mi_diario.txt ya existe en tu carpeta.
# Contenido actual:
# Nombre: Carlos
# Fecha: 2026-03-21
```

El `\n` al final de cada línea es el salto de línea. Sin él, todo el texto quedaría en una sola línea. El archivo se crea aunque no exista. Si ya existía, su contenido anterior fue borrado.

Paso 2: Leer el archivo completo con modo "r".

```
with open("mi_diario.txt", "r") as f:
    contenido = f.read()
    print(contenido)

# Salida:
# Nombre: Carlos
# Fecha: 2026-03-21
```

`.read()` devuelve TODO el contenido del archivo como un string. Si quieres leer línea por línea, puedes usar `.readlines()` (devuelve una lista de strings) o iterar directamente con `for` línea in `f`:

```
# Alternativa: leer línea por línea
with open("mi_diario.txt", "r") as f:
    for linea in f:
        print(linea.strip()) # .strip() elimina el \n del final
```

Paso 3: Añadir contenido con modo "a" sin borrar lo anterior.

```
with open("mi_diario.txt", "a") as f:
    f.write("Nota: Hoy aprendí archivos en Python\n")

# Contenido actual del archivo:
# Nombre: Carlos
```

```
# Fecha: 2026-03-21
# Nota: Hoy aprendí archivos en Python
```

Modo "a" (append) agrega al final. El archivo no se borra. Si no existe, lo crea. Es el modo seguro cuando quieres agregar registros.

Paso 4: Verificar el resultado final.

```
with open("mi_diario.txt", "r") as f:
    lineas = f.readlines()          # lista de líneas
    print(f"El archivo tiene {len(lineas)} líneas")
    for i, linea in enumerate(lineas):
        print(f"Línea {i+1}: {linea.strip()}")

# Salida:
# El archivo tiene 3 líneas
# Línea 1: Nombre: Carlos
# Línea 2: Fecha: 2026-03-21
# Línea 3: Nota: Hoy aprendí archivos en Python
```

Mini Reto 1.1

- Abre el archivo con modo "w" de nuevo y verifica (imprimiendo) que quedó vacío.
- Escribe ahora 5 líneas de texto libre usando un for y una lista de strings.
- Lee el archivo con `.readlines()` y muestra solo las líneas que contengan la letra "a".

Pista: `if 'a' in linea.strip():`

Solución sugerida — Ejercicio 1.1

```
# Paso 1: escribir
with open("mi_diario.txt", "w") as f:
    f.write("Nombre: Carlos\n")
    f.write("Fecha: 2026-03-21\n")

# Paso 2: leer
with open("mi_diario.txt", "r") as f:
    print(f.read())

# Paso 3: añadir
with open("mi_diario.txt", "a") as f:
    f.write("Nota: Hoy aprendí archivos\n")

# Mini reto
frases = ['hola mundo', 'python es genial', 'archivos de texto', 'me gusta programar', 'campuslands']
with open("mi_diario.txt", "w") as f:
    for frase in frases:
        f.write(frase + "\n")

with open("mi_diario.txt", "r") as f:
    for linea in f:
        if 'a' in linea.strip():
            print(linea.strip())
```

Explicación:

- `.read()` → string completo. `.readlines()` → lista de strings. `for linea in f:` → itera línea a línea.
- `.strip()` elimina espacios y saltos de línea al inicio y final de cada string.
- `enumerate()` te da el índice y el valor al mismo tiempo: `for i, linea in enumerate(lista):`

Ejercicio 1.2 — Registro de actividad (log)

Enunciado

Crea un programa que simule un registro de actividad. El usuario escribe acciones (hasta escribir "salir") y cada acción se guarda en un archivo "log.txt" con la hora. Al terminar, muestra el historial completo.

Guía paso a paso:

Paso 1: Importa datetime para registrar la hora.

```
from datetime import datetime

# datetime.now() devuelve la fecha y hora actual
# .strftime() la formatea como texto
hora = datetime.now().strftime("%H:%M:%S")
print(hora) # ejemplo: 14:32:05
```

Paso 2: Pide acciones al usuario y guárdalas con modo "a".

```
from datetime import datetime

print('Escribe acciones (escribe "salir" para terminar):')

while True:
    accion = input('> ')
    if accion.lower() == "salir":
        break
    hora = datetime.now().strftime("%H:%M:%S")
    entrada = f"[{hora}] {accion}\n"
    with open("log.txt", "a") as f:
        f.write(entrada)
    print(f" Guardado: {entrada.strip()}")
```

Abrir el archivo con "a" dentro del while garantiza que cada acción se añade al final sin borrar las anteriores. Se abre y cierra en cada vuelta — esto es correcto y seguro.

Paso 3: Muestra el historial completo al terminar.

```
import os

if os.path.exists("log.txt"):
    print("\n— Historial de actividad —")
    with open("log.txt", "r") as f:
        print(f.read())
else:
    print("No hay actividad registrada.")
```

`os.path.exists()` verifica si un archivo existe antes de intentar abrirlo. Así evitas el `FileNotFoundError` cuando el archivo todavía no se ha creado.

Error común que verás aquí:

```
# Si intentas leer un archivo que no existe:
with open("log.txt", "r") as f:
```

```

    print(f.read())
# → FileNotFoundError: No such file or directory

# Solución 1: verificar antes
import os
if os.path.exists("log.txt"):
    with open("log.txt", "r") as f: print(f.read())

# Solución 2: usar try/except
try:
    with open("log.txt", "r") as f: print(f.read())
except FileNotFoundError:
    print("El archivo no existe todavía.")

```

Mini Reto 1.2

- Modifica el programa para que también muestre cuántas acciones se registraron al terminar.
- Agrega una opción "borrar" que limpie el log (sin usar "w" directamente — usa `os.remove("log.txt")`).

Pista: cuenta líneas con `len(f.readlines())` al final.

Solución sugerida — Ejercicio 1.2

```

from datetime import datetime
import os

ARCHIVO = "log.txt"
contador = 0

print('Escribe acciones ("salir" para terminar, "borrar" para limpiar):')

while True:
    accion = input('> ').strip()
    if accion.lower() == "salir":
        break
    elif accion.lower() == "borrar":
        if os.path.exists(ARCHIVO):
            os.remove(ARCHIVO)
            print("Log borrado.")
        continue
    hora = datetime.now().strftime("%H:%M:%S")
    with open(ARCHIVO, "a") as f:
        f.write(f"[{hora}] {accion}\n")
    contador += 1

print(f"\nSe registraron {contador} acciones.")
if os.path.exists(ARCHIVO):
    with open(ARCHIVO, "r") as f:
        lineas = f.readlines()
        print(f"Total en archivo: {len(lineas)} líneas")
        print(f.read() if lineas else 'Archivo vacío')

```

Explicación:

- `os.path.exists()` previene el `FileNotFoundError` — verifica antes de abrir en modo lectura.
- `os.remove()` elimina el archivo del disco. Es equivalente a borrarlo desde el explorador.

- `try/except FileNotFoundError` es la alternativa profesional a `os.path.exists()`.

Tema 2: Archivos CSV (.csv)

CSV (Comma Separated Values) es el formato más usado para compartir datos tabulares — exactamente como una hoja de Excel pero en texto plano. Python tiene el módulo csv que simplifica leerlos y escribirlos.

Ejercicio 2.1 — Crear y leer una lista de estudiantes

Enunciado

Crema un archivo "estudiantes.csv" con columnas: nombre, edad, nota. Agrega al menos 4 estudiantes. Luego léelo con DictReader y muestra cada estudiante como un diccionario. Finalmente calcula el promedio de notas.

Guía paso a paso:

Paso 1: Escribe el CSV con writer().

```
import csv

estudiantes = [
    ["nombre", "edad", "nota"],  # primera fila = encabezados
    ["Ana", 20, 4.5],
    ["Luis", 19, 3.2],
    ["María", 21, 4.8],
    ["Carlos", 20, 2.9]
]

with open("estudiantes.csv", "w", newline="") as f:
    escritor = csv.writer(f)
    escritor.writerows(estudiantes)  # escribe todas las filas de una vez

# Archivo generado:
# nombre,edad,nota
# Ana,20,4.5
# Luis,19,3.2
# María,21,4.8
# Carlos,20,2.9
```

¿Por qué `newline=""`? En Windows, sin `newline=""` cada fila del CSV tiene una línea en blanco extra entre ellas. Es un comportamiento del sistema operativo que el módulo csv necesita controlar internamente.

Paso 2: Lee el CSV con DictReader (cada fila como diccionario).

```
import csv

with open("estudiantes.csv", "r") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila)  # cada fila es un diccionario

# Salida:
# {'nombre': 'Ana', 'edad': '20', 'nota': '4.5'}
# {'nombre': 'Luis', 'edad': '19', 'nota': '3.2'}
# {'nombre': 'María', 'edad': '21', 'nota': '4.8'}
```

```
# {'nombre': 'Carlos', 'edad': '20', 'nota': '2.9'}
```

Importante: DictReader convierte TODOS los valores a string. El 4.5 de la nota llega como "4.5" (texto). Para hacer cálculos necesitas convertirlo: `float(fila["nota"])`.

Paso 3: Calcula el promedio convirtiendo los valores.

```
import csv

suma = 0
total = 0

with open("estudiantes.csv", "r") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        nota = float(fila["nota"]) # convierte string a float
        suma += nota
        total += 1
        estado = "Aprobado" if nota >= 3.0 else "Reprobado"
        print(f"{fila['nombre']}: {nota} → {estado}")

print(f"\nPromedio del grupo: {suma/total:.2f}")
```

Mini Reto 2.1

- Agrega un quinto estudiante al CSV con modo "a" usando `csv.writer`. Verifica que se añade al final sin borrar los demás.
- Lee el archivo completo de nuevo y muestra solo los estudiantes con `nota >= 4.0`.

Pista: abre con "a", crea `csv.writer(f)` y usa `.writerow(["Nombre", edad, nota])`.

Solución sugerida — Ejercicio 2.1

```
import csv

# Escribir
datos = [
    ['nombre', 'edad', 'nota'], ['Ana', 20, 4.5], ['Luis', 19, 3.2], ['María', 21, 4.8], ['Carlos', 20, 2.9]
]
with open("estudiantes.csv", "w", newline="") as f:
    csv.writer(f).writerows(datos)

# Añadir un estudiante
with open("estudiantes.csv", "a", newline="") as f:
    csv.writer(f).writerow(["Sofía", 22, 4.1])

# Leer y filtrar
suma = total = 0
with open("estudiantes.csv", "r") as f:
    for fila in csv.DictReader(f):
        nota = float(fila['nota'])
        suma += nota; total += 1
        if nota >= 4.0:
            print(f"{fila['nombre']}: {nota}")
print(f"Promedio: {suma/total:.2f}")
```

Explicación:

- DictReader usa la primera fila como encabezados automáticamente.
- Al añadir con "a", el nuevo estudiante va al final — sin encabezados repetidos.
- Siempre convierte los números: int() o float() según el tipo de dato.

Ejercicio 2.2 — Reporte de ventas con DictWriter

Enunciado

Tienes una lista de diccionarios con ventas por vendedor. Guárdalos en un CSV con DictWriter. Luego léelos, calcula el total vendido por cada vendedor y genera un reporte que muestre quién vendió más.

Guía paso a paso:

Paso 1: Escribe con DictWriter (desde diccionarios).

```
import csv

ventas = [
    {"vendedor": "Ana",      "producto": "Menta",      "cantidad": 50, "precio": 300},
    {"vendedor": "Luis",     "producto": "Chocorramo",  "cantidad": 20, "precio": 1000},
    {"vendedor": "Ana",      "producto": "Esfero",      "cantidad": 30, "precio": 2700},
    {"vendedor": "María",    "producto": "Chocolatina", "cantidad": 40, "precio": 2500},
    {"vendedor": "Luis",     "producto": "Menta",      "cantidad": 80, "precio": 300},
]

campos = ["vendedor", "producto", "cantidad", "precio"]

with open("ventas.csv", "w", newline="") as f:
    escritor = csv.DictWriter(f, fieldnames=campos)
    escritor.writeheader()      # escribe los encabezados
    escritor.writerows(ventas)  # escribe todas las filas
```

Paso 2: Lee el CSV y agrupa totales por vendedor.

```
totales = {} # diccionario para acumular por vendedor

with open("ventas.csv", "r") as f:
    for fila in csv.DictReader(f):
        vendedor = fila["vendedor"]
        subtotal = int(fila["cantidad"]) * int(fila["precio"])
        if vendedor in totales:
            totales[vendedor] += subtotal
        else:
            totales[vendedor] = subtotal

print('— Reporte de ventas —')
mejor_vendedor = ''
mejor_total = 0
for vendedor, total in totales.items():
```

```

print(f"{vendedor}: ${total:,}")
if total > mejor_total:
    mejor_total = total
    mejor_vendedor = vendedor

print(f"\nMejor vendedor: {mejor_vendedor} (${mejor_total:,})")

```

Mini Reto 2.2

- Añade el campo 'subtotal' a cada fila del CSV (cantidad × precio) usando DictWriter.
- Lee de nuevo y genera un reporte donde se vea el subtotal de cada transacción.

Pista: agrega 'subtotal' a los campos y calcula antes de escribir cada fila.

Solución sugerida — Ejercicio 2.2

```

import csv

ventas = [
    {"vendedor": "Ana", "producto": "Menta", "cantidad": 50, "precio": 300},
    {"vendedor": "Luis", "producto": "Chocorramo", "cantidad": 20, "precio": 1000},
    {"vendedor": "Ana", "producto": "Esfero", "cantidad": 30, "precio": 2700},
    {"vendedor": "María", "producto": "Chocolatina", "cantidad": 40, "precio": 2500},
    {"vendedor": "Luis", "producto": "Menta", "cantidad": 80, "precio": 300},
]

# Añadir subtotal
for v in ventas:
    v["subtotal"] = v["cantidad"] * v["precio"]

campos = ["vendedor", "producto", "cantidad", "precio", "subtotal"]
with open("ventas.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=campos)
    w.writeheader(); w.writerows(ventas)

totales = {}
with open("ventas.csv", "r") as f:
    for fila in csv.DictReader(f):
        v = fila["vendedor"]
        totales[v] = totales.get(v, 0) + int(fila["subtotal"])

for v, t in totales.items(): print(f'{v}: ${t:,}')
print('Mejor:', max(totales, key=totales.get))

```

Explicación:

- DictWriter.writeheader() escribe los nombres de columna automáticamente.
- totales.get(v, 0) es más limpio que el if/else para acumular en un diccionario.
- max(diccionario, key=diccionario.get) encuentra la key con el valor más alto.

Tema 3: Archivos JSON (.json)

JSON es el formato estándar para transmitir datos entre sistemas y APIs web. En Python, trabajar con JSON es natural porque su estructura es casi idéntica a los diccionarios y listas.

Ejercicio 3.1 — Guardar y cargar un catálogo de productos

Enunciado

Crea un catálogo de productos como lista de diccionarios. Guárdalo en "catalogo.json". Luego cárgalo de nuevo, muestra todos los productos y filtra los que cuestan menos de \$2000.

Guía paso a paso:

Paso 1: Crea el catálogo y guárdalo con `json.dumps()`.

```
import json

catalogo = [
    {"nombre": "Menta", "precio": 300, "stock": 50},
    {"nombre": "Chocorramo", "precio": 1000, "stock": 30},
    {"nombre": "Esfero", "precio": 2700, "stock": 20},
    {"nombre": "Chocolatina", "precio": 2500, "stock": 15},
    {"nombre": "Cuaderno", "precio": 4500, "stock": 8}
]

with open("catalogo.json", "w") as f:
    f.write(json.dumps(catalogo, indent=2, ensure_ascii=False))

# El archivo catalogo.json contiene:
# [
#   {
#     "nombre": "Menta",
#     "precio": 300,
#     "stock": 50
#   },
#   ...
# ]
```

`indent=2` agrega sangría para que el JSON sea legible por humanos. Sin él, todo queda en una sola línea.

`ensure_ascii=False` permite que las tildes y la ñ se guarden correctamente.

Paso 2: Carga el JSON con `json.load()` y muestra los productos.

```
import json

with open("catalogo.json", "r") as f:
    productos = json.load(f) # convierte JSON → lista de diccionarios

print(f"Catálogo: {len(productos)} productos\n")
for p in productos:
    print(f"  {p['nombre']:<15} ${p['precio']:>6,}   Stock: {p['stock']}")
```

```
# Salida:
# Catálogo: 5 productos
#   Menta          $    300   Stock: 50
#   Chocorramo     $  1,000   Stock: 30
#   Esfero         $  2,700   Stock: 20
#   Chokolatina    $  2,500   Stock: 15
#   Cuaderno       $  4,500   Stock: 8
```

`json.load()` lee desde un ARCHIVO y devuelve el objeto Python (lista o diccionario).

`json.loads()` lee desde un STRING. Confundirlos es el error más común con JSON. La "s" al final = string.

Paso 3: Filtra productos baratos.

```
baratos = [p for p in productos if p['precio'] < 2000]
print(f"\nProductos menores a $2.000:")
for p in baratos:
    print(f"    {p['nombre']}: ${p['precio']}")
```

Mini Reto 3.1

- Modifica el stock de 'Menta' a 99 directamente en el diccionario cargado y guarda el JSON de nuevo.
- Calcula el valor total en bodega de cada producto (precio × stock) y agrégalo como campo 'valor_bodega'.

Pista: carga el JSON, modifica el diccionario con Python y vuelve a escribirlo.

Solución sugerida — Ejercicio 3.1

```
import json

# Guardar
catalogo = [
    {"nombre": "Menta", "precio": 300, "stock": 50},
    {"nombre": "Chocorramo", "precio": 1000, "stock": 30},
    {"nombre": "Esfero", "precio": 2700, "stock": 20},
    {"nombre": "Chokolatina", "precio": 2500, "stock": 15},
    {"nombre": "Cuaderno", "precio": 4500, "stock": 8}
]
with open("catalogo.json", "w") as f:
    f.write(json.dumps(catalogo, indent=2, ensure_ascii=False))

# Cargar y modificar
with open("catalogo.json", "r") as f:
    productos = json.load(f)

for p in productos:
    if p["nombre"] == "Menta":
        p["stock"] = 99
        p["valor_bodega"] = p["precio"] * p["stock"]

# Guardar con cambios
with open("catalogo.json", "w") as f:
    f.write(json.dumps(productos, indent=2, ensure_ascii=False))

# Mostrar
```

```
for p in productos:
    print(f'{p["nombre"]}: stock={p["stock"]} | bodega=${p["valor_bodega"]},,}')"
```

Explicación:

- El flujo de modificación siempre es: cargar → modificar en Python → guardar de nuevo.
- `json.load()` devuelve objetos Python normales — puedes modificarlos con la misma sintaxis de listas y diccionarios.
- Para guardar los cambios, simplemente sobrescribe el archivo con "w".

Ejercicio 3.2 — Sistema de usuarios en JSON

Enunciado

Crea un sistema que permita registrar usuarios en un archivo JSON. El programa debe: cargar usuarios existentes, agregar nuevos, buscar por nombre y guardar los cambios. El archivo "usuarios.json" persiste entre ejecuciones.

Guía paso a paso:

Paso 1: Función para cargar usuarios (con manejo de archivo inexistente).

```
import json, os

ARCHIVO = 'usuarios.json'

def cargar_usuarios():
    if not os.path.exists(ARCHIVO):
        return [] # si no existe, retorna lista vacía
    with open(ARCHIVO, 'r') as f:
        return json.load(f)

def guardar_usuarios(usuarios):
    with open(ARCHIVO, 'w') as f:
        f.write(json.dumps(usuarios, indent=2, ensure_ascii=False))
```

Separar las operaciones de archivo en funciones es una buena práctica. El resto del programa llama a `cargar_usuarios()` y `guardar_usuarios()` sin preocuparse por los detalles del archivo.

Paso 2: Función para agregar un usuario.

```
def agregar_usuario(usuarios, nombre, correo, edad):
    # Verificar que el correo no esté duplicado
    for u in usuarios:
        if u['correo'] == correo:
            print(f'El correo {correo} ya está registrado.')
            return usuarios
    nuevo = {'nombre': nombre, 'correo': correo, 'edad': edad}
    usuarios.append(nuevo)
    print(f'Usuario {nombre} registrado exitosamente.')
    return usuarios
```

Paso 3: Función para buscar usuario.

```
def buscar_usuario(usuarios, nombre):
    for u in usuarios:
```



```

        if u['nombre'].lower() == nombre.lower():
            return u
    return None

```

Paso 4: Programa principal que usa las funciones.

```

usuarios = cargar_usuarios()
print(f'Usuarios cargados: {len(usuarios)}')

# Agregar usuarios
usuarios = agregar_usuario(usuarios, 'Ana', 'ana@mail.com', 25)
usuarios = agregar_usuario(usuarios, 'Luis', 'luis@mail.com', 30)
usuarios = agregar_usuario(usuarios, 'María', 'ana@mail.com', 22) # correo
duplicado

guardar_usuarios(usuarios)

# Buscar
encontrado = buscar_usuario(usuarios, 'ana')
if encontrado:
    print(f'Encontrado: {encontrado}')
else:
    print('Usuario no encontrado.')

```

Mini Reto 3.2

- Agrega una función `eliminar_usuario(usuarios, correo)` que elimine al usuario con ese correo.
- Ejecuta el programa dos veces seguidas. ¿Los usuarios del primer run persisten en el segundo?

Pista: `usuarios = [u for u in usuarios if u['correo'] != correo]`

Solución sugerida — Ejercicio 3.2

```

import json, os
ARCHIVO = 'usuarios.json'

def cargar_usuarios():
    if not os.path.exists(ARCHIVO): return []
    with open(ARCHIVO, 'r') as f: return json.load(f)

def guardar_usuarios(u):
    with open(ARCHIVO, 'w') as f:
        f.write(json.dumps(u, indent=2, ensure_ascii=False))

def agregar_usuario(usuarios, nombre, correo, edad):
    for u in usuarios:
        if u['correo'] == correo:
            print(f'Correo duplicado: {correo}'); return usuarios
    usuarios.append({'nombre': nombre, 'correo': correo, 'edad': edad})
    print(f'{nombre} registrado.');
```

```
usuarios = agregar_usuario(usuarios, 'Luis', 'luis@mail.com', 30)
usuarios = eliminar_usuario(usuarios, 'luis@mail.com')
guardar_usuarios(usuarios)
print(f'Usuarios guardados: {len(usuarios)}')
```

Explicación:

- Separar en funciones hace el código mantenible: si el formato del archivo cambia, solo editas cargar/guardar.
- La persistencia entre ejecuciones es la principal ventaja de los archivos: ejecuta el programa dos veces y verás que los usuarios del primer run siguen ahí.
- La list comprehension [u for u in lista if condicion] es la forma más limpia de filtrar/eliminar.

PARTE 2: EJERCICIOS INDEPENDIENTES

Practica lo aprendido sin guía — Solo enunciado y datos de prueba

Archivos de Texto (.txt)

Ejercicio P1 — Agenda de tareas

Enunciado

Crea un programa de lista de tareas. El usuario puede: agregar tareas (se guardan en "tareas.txt" con modo "a"), ver todas las tareas (leer e imprimir), y marcar una tarea como hecha (leer todas, modificar la que corresponde y reescribir con "w"). Usa un menú con while.

Ejemplo de uso	Salida esperada
Opción 1: agregar 'Estudiar Python'	[1] Estudiar Python
Opción 2: ver tareas	[1] Estudiar Python[2] Hacer ejercicio

Pista: Para marcar como hecha, lee con `readlines()`, modifica el elemento de la lista y reescribe todo con "w".

Ejercicio P2 — Contador de palabras en archivo

Enunciado

Pide al usuario que ingrese un párrafo de texto. Guárdalo en "texto.txt". Luego léelo, cuenta cuántas veces aparece cada palabra (ignora mayúsculas y signos de puntuación) y muestra las 5 palabras más frecuentes.

Ejemplo de uso	Salida esperada
Párrafo: 'Python es genial. Python es fácil.'	python: 2es: 2genial: 1fácil: 1

Pista: Para limpiar: `palabra.lower().strip(".,!?:;")`. Usa un diccionario para contar. Ordena con `sorted(conteo.items(), key=lambda x: x[1], reverse=True)`.

Ejercicio P3 — Backup con fecha

Enunciado

Crea un programa que lea un archivo "notas.txt" y haga una copia de respaldo con la fecha en el nombre, por ejemplo "notas_2026-03-21.txt". Si el archivo original no existe, créalo con contenido de prueba primero.

Ejemplo de uso	Salida esperada
notas.txt existe con contenido	Backup creado: notas_2026-03-21.txt

Pista: Usa `datetime.now().strftime("%Y-%m-%d")` para la fecha. Lee el original y escribe el backup en un archivo con nombre diferente.

Archivos CSV (.csv)

Ejercicio P4 — Inventario de tienda

Enunciado

Crea un CSV "inventario.csv" con columnas: producto, precio, stock. Implementa estas operaciones con funciones: (1) ver inventario completo, (2) agregar producto nuevo, (3) actualizar stock de un producto existente, (4) mostrar productos con stock menor a 10.

Ejemplo de uso	Salida esperada
Opción 3: actualizar stock de 'Menta' a 80	Stock actualizado. Menta ahora tiene 80 unidades.

Pista: Para actualizar: carga todos los datos, modifica el diccionario correspondiente y reescribe todo el CSV con DictWriter.

Ejercicio P5 — Importar notas de estudiantes

Enunciado

Tienes un CSV "notas.csv" con columnas nombre, nota1, nota2, nota3. Lee el archivo con DictReader, calcula el promedio de cada estudiante y escribe un nuevo CSV "resultados.csv" con columnas nombre, promedio, estado (Aprobado/Reprobado).

Ejemplo de uso	Salida esperada
notas.csv: Ana,4.5,3.8,4.0	resultados.csv: Ana,4.1,Aprobado

Pista: Calcula promedio = $(\text{float}(\text{nota1}) + \text{float}(\text{nota2}) + \text{float}(\text{nota3})) / 3$. Usa DictWriter para resultados.csv con `fieldnames=["nombre", "promedio", "estado"]`.

Archivos JSON (.json)

Ejercicio P6 — Configuración de aplicación

Enunciado

Crea un sistema de configuración que guarde preferencias en "config.json" (idioma, tema, tamaño_fuente). Si el archivo no existe, usa valores por defecto. El usuario puede cambiar cualquier preferencia y los cambios persisten entre ejecuciones.

Ejemplo de uso	Salida esperada
Primera ejecución (sin config.json)	Config creada con valores por defecto.
Segunda ejecución	Config cargada: tema=oscuro, idioma=es

Pista: Usa `os.path.exists()` para verificar. `config = cargar_config()` → mostrar → modificar → `guardar_config(config)`.

Ejercicio P7 — Historial de ventas acumulado

Enunciado

Crea un programa que registre ventas en "historial.json". Cada venta es un diccionario con: producto, cantidad, precio, fecha y total (cantidad × precio). El programa agrega nuevas ventas al historial existente y muestra el total acumulado de todas las ventas.

Ejemplo de uso	Salida esperada
Venta: Menta, 10 unidades, \$300	Venta guardada. Total histórico: \$3.000

Pista: Carga el JSON existente (o lista vacía si no existe), haz `.append()` con la nueva venta y guarda todo de nuevo.

Ejercicio Final — Sistema de Gestión Multi-Archivo

Enunciado

Construye un sistema que use los tres formatos aprendidos al mismo tiempo.

El sistema debe:

1. Guardar el catálogo de productos en "catalogo.json" (nombre, precio, stock).
2. Registrar cada venta realizada en "ventas.csv" (producto, cantidad, total, fecha).
3. Generar un reporte en "reporte.txt" al cerrar el programa con: total vendido del día, producto más vendido y productos con stock bajo.

Restricciones:

- El catálogo persiste entre ejecuciones (JSON).
- Las ventas se acumulan en el CSV con modo "a".
- El reporte se genera con modo "w" al cerrar (sobreescribe el del día anterior).
- Si el stock de un producto llega a 0, no permite venderlo.

Pista: Usa funciones separadas: `cargar_catalogo()`, `registrar_venta()`, `generar_reporte()`. El programa principal es un `while True` con un menú de opciones.

¡Felicidades por completar los ejercicios!

"Los datos sin guardar son como pensamientos sin escribir: se pierden"

Ahora puedes guardar, leer y compartir datos en cualquier formato.
.txt para texto libre. .csv para tablas. .json para estructuras. Úsalos juntos.